RLab 06 – Multiple Linear Regression Model

Name

Invalid Date

Introduction

We've seen a review of simple linear regression and what it can do. In a real-world setting, it's often unreasonable to look for a simple relationship between one explanatory and one response variable. There are often many explanatory variables that may contribute to predicting a response variable. Multiple linear regression gives one way to look at the relationship between those multiple explanatory variables and the response variable.

In this lab, we'll look at the data set on faculty salaries one more time. This time we'll really try to get an answer to the question of whether faculty salaries are related to `sex`, our male/female variable. This might be the *first* step in establishing actual discrimination, but remember that there are more variables that are not included in this data set that might be related to salary, and even if a statistical relationship is shown, there are more steps to prove that discrimination actually occurred.

First we'll load the `carData` package that contains the `Salaries` data set, as well as our other favorite packages.

```r
library(carData)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

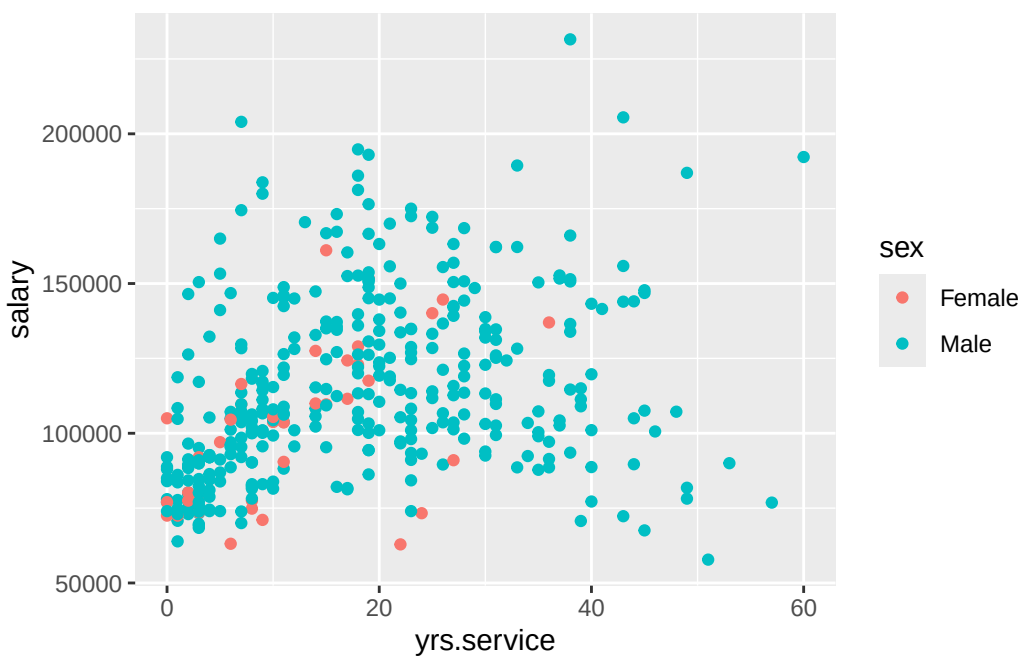
```r
library(modelr)
library(ggplot2)
# install.packages("cvTools")
library(cvTools)
```

```
Loading required package: lattice
Loading required package: robustbase

Attaching package: 'cvTools'

The following object is masked from 'package:modelr':

    mape
```

```r
set.seed(92425)
```

Remember that there were no female faculty members with more than about 40 years of service, so we'll also cut the data set down to only those with less than 40 years so that we're comparing apples to apples. The data frame `sal40` will be our new data frame.

```r
ggplot(Salaries) +
  geom_point(mapping = aes(x=yrs.service, y=salary, color=sex))
```

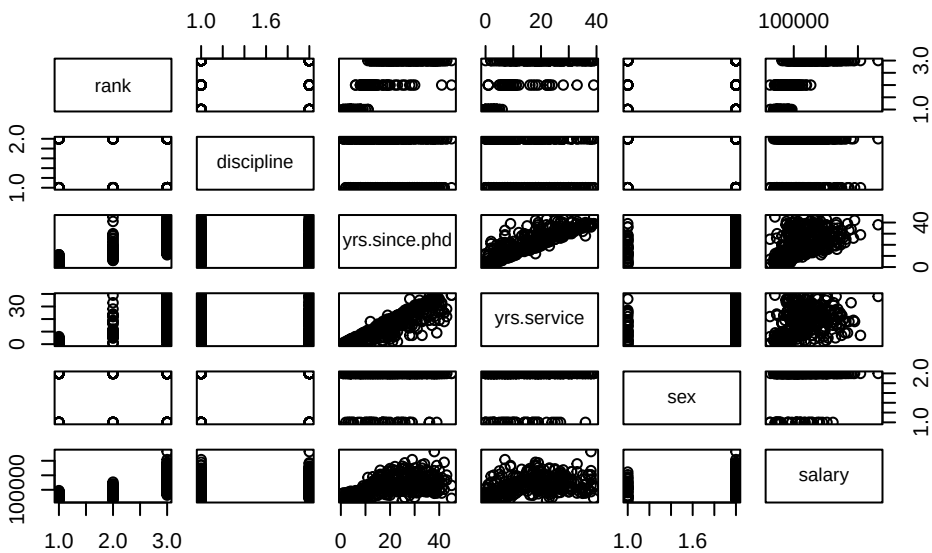


```r
sal40 <- Salaries %>% filter(yrs.service < 40)
```

The Big Picture

We've done this before, but here is a reminder of the big picture. The `pairs` command creates lots of scatter plots that show the relationship between each variable, and the `cor` command gives linear correlation coefficients.

```r
pairs(sal40)
```




```r
# cor can only take numeric variables.
sal40 %>% select(yrs.since.phd, yrs.service, salary) %>% cor()
```

```
              yrs.since.phd yrs.service    salary
yrs.since.phd     1.0000000   0.8794509 0.5129385
yrs.service       0.8794509   1.0000000 0.4186644
salary            0.5129385   0.4186644 1.0000000
```

You can see that `yrs.since.phd` has a slightly stronger linear relationship to salary than does `yrs.service`.

Splitting our data

One method of checking for overfitting is to divide our data into two: a "train" set that we'll use to create (or train) a model, and a "test" set that we'll use to check it later. We use the set.seed command so that we all make the same random split. Both sample_frac and anti_join are dplyr commands.

```
set.seed(20190407)
sal40_train <- sal40 %>% sample_frac(.65)
sal40_test  <- anti_join(sal40, sal40_train)
```

```
Joining with `by = join_by(rank, discipline, yrs.since.phd, yrs.service, sex,
salary)`
```

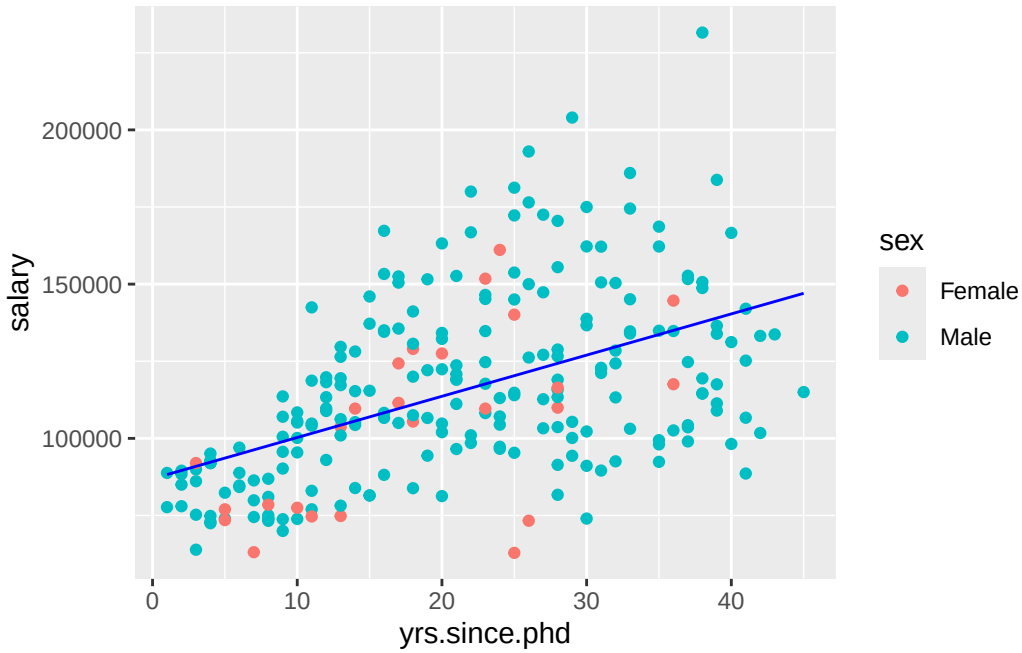
Simple Linear Regression

Again, a review. If we want to find the best linear relationship between `yrs.since.phd` and `salary`, the `lm` command does the trick. We can look at the numeric output and plot predictions and residuals.

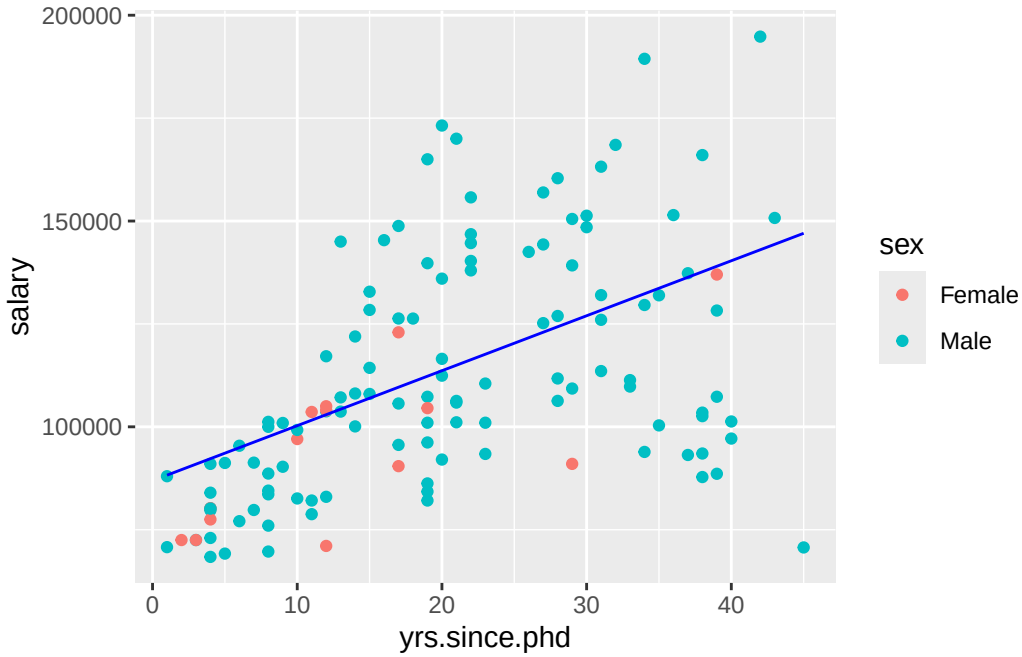
```
fit.ysp <- lm(salary ~ yrs.since.phd, data=sal40_train)
summary(fit.ysp)
```

```
Call:
lm(formula = salary ~ yrs.since.phd, data = sal40_train)

Residuals:
   Min     1Q Median     3Q    Max 
-57427 -19108  -1652  14657  93873 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)    86923.9     3518.9   24.70   <2e-16 ***
yrs.since.phd   1335.5      147.2    9.07   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 25810 on 239 degrees of freedom
Multiple R-squared:  0.2561,	Adjusted R-squared:  0.253 
F-statistic: 82.26 on 1 and 239 DF,  p-value: < 2.2e-16
```

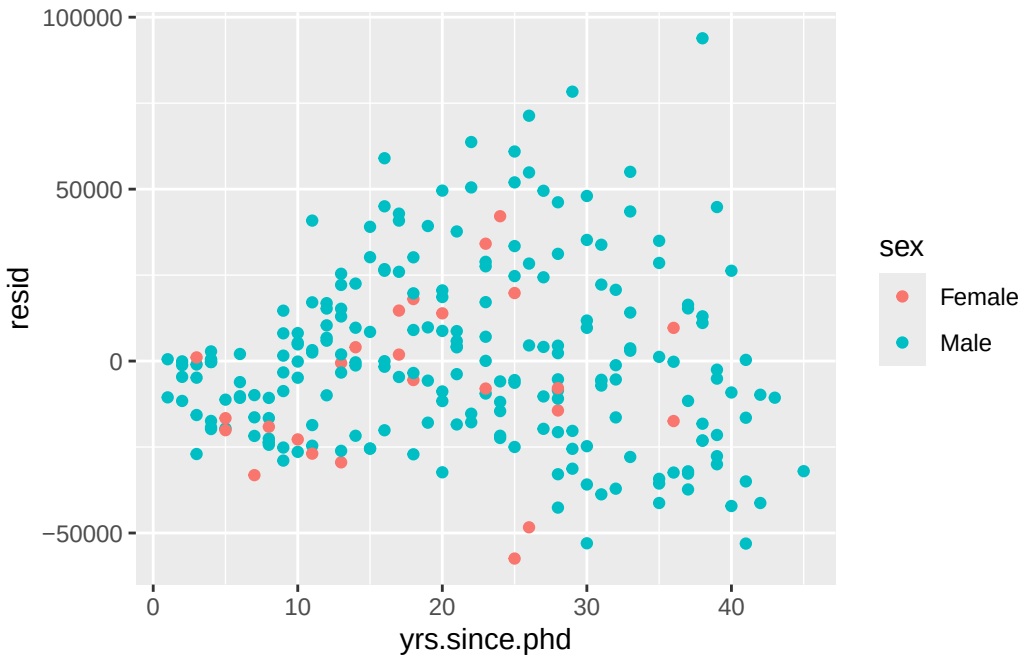
```r
sal40_train %>% add_predictions(fit.ysp) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=salary, color=sex)) +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred), color="blue")
```



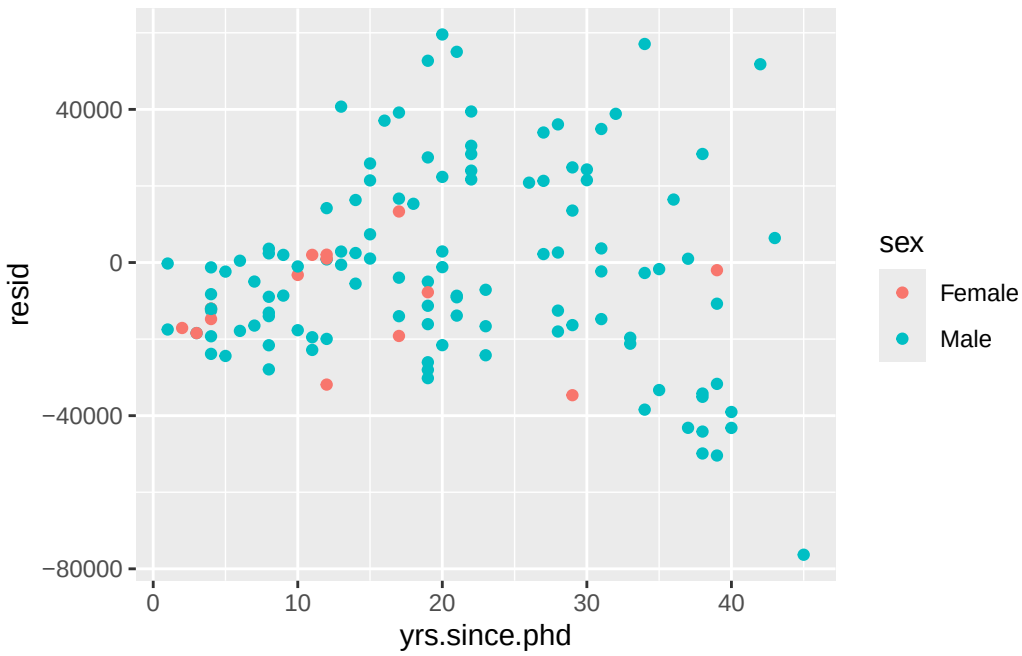

```r
sal40_test %>% add_predictions(fit.ysp) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=salary, color=sex)) +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred), color="blue")
```




```
sal40_train %>%  add_residuals(fit.ysp) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=resid, color=sex))
```



```
sal40_test %>%  add_residuals(fit.ysp) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=resid, color=sex))
```

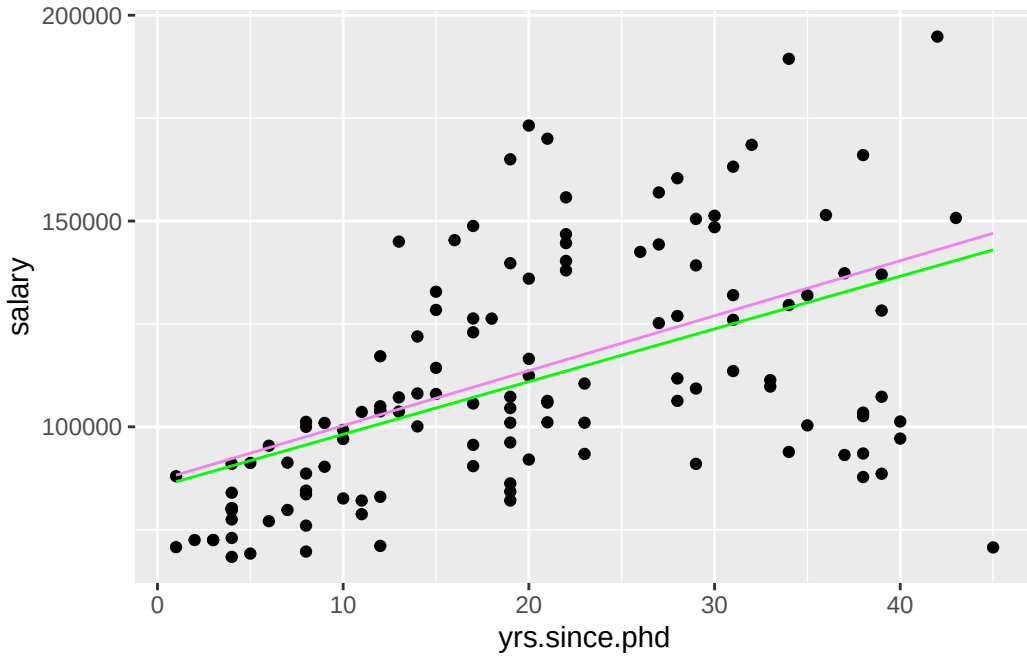


Let's take a look at the difference between the regression line built using training subset and the regression line built using testing subset.

```
fit.ysp_test <- lm(salary ~ yrs.since.phd, data=sal40_test)
coef(fit.ysp_test)
```

```
  (Intercept) yrs.since.phd
    85385.489      1280.268
```

```
sal40_test_expanded <- sal40_test %>% add_predictions(fit.ysp) %>% add_predictions(fit.ysp_te

ggplot(data = sal40_test_expanded) +
  geom_point(mapping = aes(x=yrs.since.phd, y=salary)) +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred), color="violet") +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred_test), color="green")
```



Question 1

Which one of the two lines on the graph above has smaller slope – the one build using training data subset or the one built using testing data subset? Substantiate your answer with the appropriate numbers from the model estimation outputs.

Answer to Question 1

The line built using the testing data subset has a smaller slope. Looking at the coefficients: fit.ysp (training) has a slope of 1335, while fit.ysp_test (testing) has a slope of 1280. The training model has the steeper slope, with 1335 compared to 1280 for the testing model.

Multiple Variables

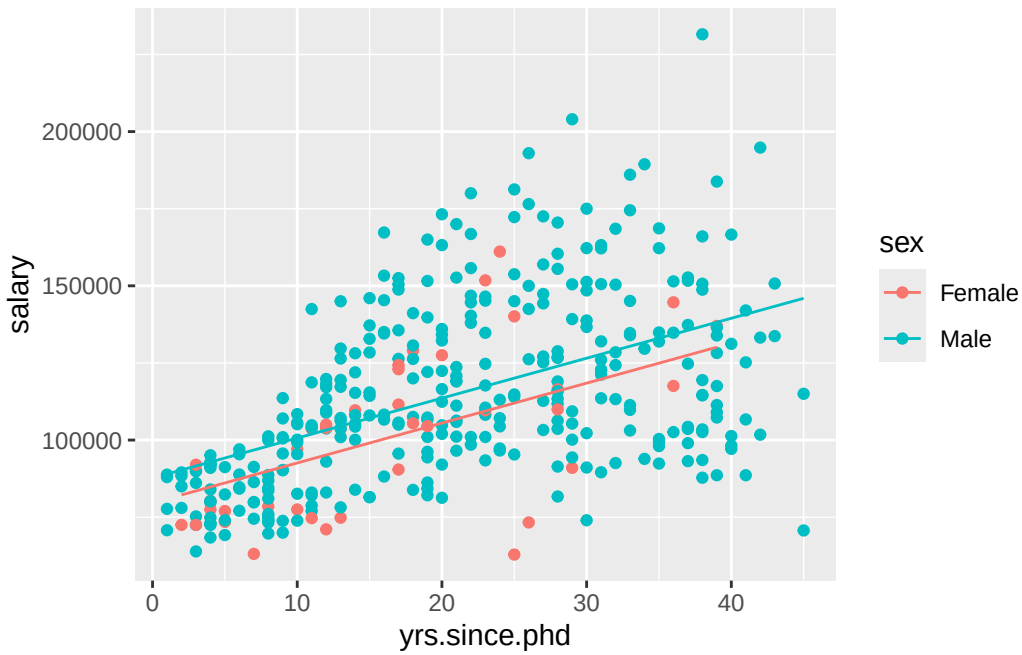
Additive (Independent) Variable Contributions

If our goal is to see whether `sex` is related to salaries, we should add it to our model! You can add explanatory variables to your model formula using the "+" sign on the right-hand-side of the formula. Both numeric and categorical (factor) variables can be used.

```
lm(response ~ exp1 + exp2 + exp3, data = data frame)
```

Using the "+" sign tells R that you expect the contributions from each variable to be *independent of each other*. In other words, you would be saying that you expect `sex` to contribute a constant amount to salary, regardless of `yrs.since.phd`. Let's see what that looks like. Graphing the results of the prediction is as easy as adding a `color` mapping to the `geom_line` command.

```
fit.ysp_sex <- lm(salary ~ yrs.since.phd + sex, data=sal40)
sal40 %>% add_predictions(fit.ysp_sex) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=salary, color=sex)) +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred, color=sex))
```



Let's look at the numeric output:

```
summary(fit.ysp_sex)
```

```
Call:
lm(formula = salary ~ yrs.since.phd + sex, data = sal40)

Residuals:
   Min     1Q Median     3Q    Max
-75249 -18038  -1776  14977  94656
```

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    79630.2     4471.6  17.808   <2e-16 ***
yrs.since.phd   1294.3      115.5  11.202   <2e-16 ***
sexMale         8076.3     4307.4   1.875   0.0616 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 25260 on 368 degrees of freedom
Multiple R-squared:  0.2701,    Adjusted R-squared:  0.2661
F-statistic: 68.08 on 2 and 368 DF,  p-value: < 2.2e-16
```

If you look in the `Coefficients` section of the output, the column marked `Pr(>|t|)` gives the p-value for that variable in the model. Roughly speaking, that p-value answers the question, "**After all other variables are accounted for in the model, does *this* variable contribute significantly to predicting the response variable?**" The lower the p-value, the **more** significant the variable is (and we often might choose a cut-off of 0.05 to denote statistical significance).

In this case, the p-value of 0.0616 indicates that `sex` is not statistically significant at the 5% significance level, although we might say that `sex` is moderately significant in predicting `salary` after `yrs.since.phd` is taken into consideration.

Note 1: If you take a linear regression class, you'll learn much more about these hypothesis tests, including the fact that you have to be cautious in interpreting them, especially when the data doesn't satisfy certain conditions. At this point, it would be dangerous to give this p-value too much weight.

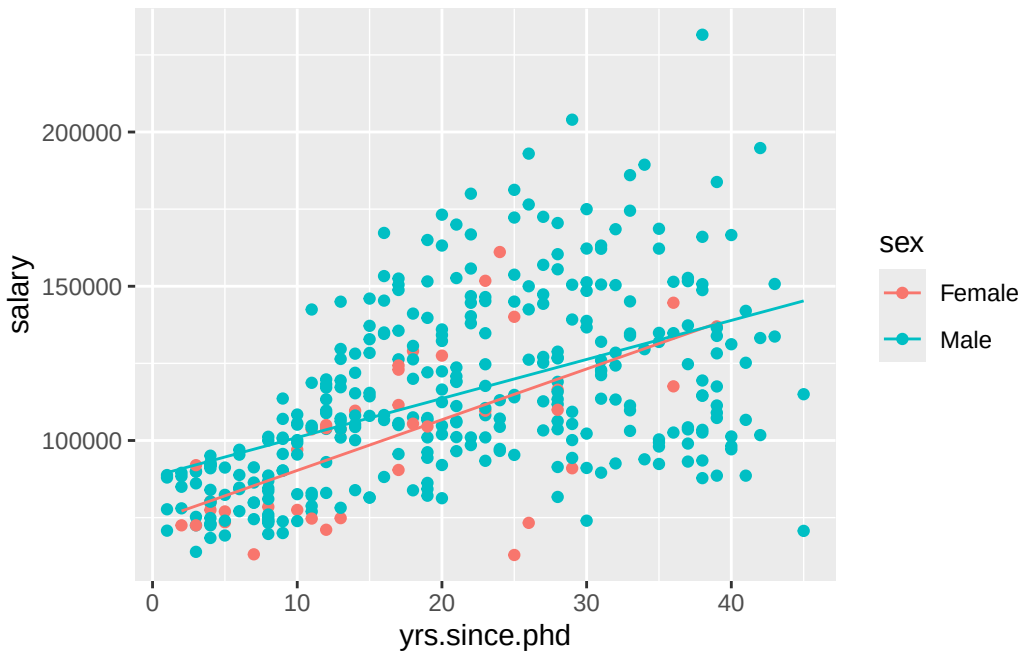
Note 2: Why isn't there a `sexFemale` line in the output of the `lm` command? The two values of `sex` are related. If you aren't "Male", you must be "Female" (at least in this data set). In such a situation, R will take only one of the two values as the basis for the test.

Interaction Terms

Sometimes you don't want to assume that variables contribute independently to predicting the response variable. For example, perhaps `sex` makes more of a difference for new PhD's than for those who have been around a while. To model that kind of relationship between variables, you can use the "*" operator, rather than the "+" operator.

In practice, most analysis of big data sets doesn't use interaction terms, because they really slow down the analysis, and it gets way too complicated. With a small number of variables, you can afford to consider interactions, however. Here's what it looks like:

```
fit.int <- lm(salary ~ yrs.since.phd * sex, data=sal40)
sal40 %>% add_predictions(fit.int) %>% ggplot() +
  geom_point(mapping = aes(x=yrs.since.phd, y=salary, color=sex)) +
  geom_line(mapping = aes(x=yrs.since.phd, y=pred, color=sex))
```



Notice that the fit lines are now allowed to have different slopes. That's the effect of the interaction term.

What is important to know about interactions, even when we don't use them formally is that, when variables are correlated with each other, adding or removing a correlated variable to a model can do wacky things. Multiple regression worries about this a bit, but wackiness still ensues.

All the Variables

We want to predict salary using all the information at our disposal. Would it simply be better to use *all* the variables? You can either list them all in the formula explicitly, or you can use "." to represent all the variables. Both of these are equivalent:

```
#   fit.all <- lm(salary ~ rank + discipline + yrs.since.phd + yrs.service + sex, data=sal40)
fit.all <- lm(salary ~ . , data=sal40)
summary(fit.all)
```

```
Call:
lm(formula = salary ~ ., data = sal40)

Residuals:
   Min     1Q Median     3Q    Max 
-46652 -13018  -1471   9583  98233 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)    65672.9     4297.1  15.283  < 2e-16 ***
rankAssocProf  11757.5     3929.5   2.992  0.00296 ** 
rankProf       42310.9     4215.4  10.037  < 2e-16 ***
disciplineB    13419.8     2245.5   5.976 5.45e-09 ***
yrs.since.phd    696.9      228.8   3.046  0.00248 ** 
yrs.service     -514.3      210.0  -2.450  0.01477 *  
sexMale         4969.4     3592.0   1.383  0.16737    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 20970 on 364 degrees of freedom
Multiple R-squared:  0.5021,	Adjusted R-squared:  0.4939 
F-statistic: 61.17 on 6 and 364 DF,  p-value: < 2.2e-16
```

Note that something (perhaps) unexpected has happened here. The p-value for `sex` is now 0.16737. After all other variables are taken into account, there is very little evidence that `sex` is useful in predicting salary. We should take this with a grain of salt, however, because some of the conditions for valid inference are not satisfied in this example. However, even if conditions were satisfied, it is likely that overall high variability in salary and relatively small difference between male and female would make it difficult to show that `sex` is statistically significant.

Also note how R has treated the other categorical variables. Rank, which has three values, has been split into two variables. `rankAssocProf` is 1 when the faculty member is an associate professor and 0 otherwise, and `rankProf` is 1 when the faculty member is a full professor and 0 otherwise. No variable is needed for assistant professor because that case is covered when the other two variables are 0.

It's hard to visualize a multiple linear regression model, so in the next section we'll talk about other ways to evaluate which model is best.

Model Quality

Akaike Information Criterion (AIC)

It might seem like a model that uses all variables would be best, but using too many variables is not a good thing, due to "overfitting." In short, when you put too many variables in a model, you may be giving too much weight to individual data points, rather than thinking of the individuals as representative of the larger population.

In regression classes, there is a tool called "Adjusted R^2," and it used to be the standard statisticians used to compare models. It is nice because it is based on normal R^2, which is commonly used across statistics. It does not penalize complicated models very much, so it still often leads to large models. In Data Science, our models can grow to be so large that we really want to get the smallest model we can, even if it might not be quite as good as a larger one.

In much of Data Science, the Akaike Information Criterion (AIC) is used instead of adjusted R^2. The AIC rewards simple models more than traditional techniques. One confusing bit is that having a higher adjusted R^2 implies that the model is better, but having a low AIC means that the model is better. Statisticians sometimes worry that the AIC only gives a relative value, so while it picks the best model, it doesn't tell you if that model is very good (unlike Adjusted R^2, for instance). Like with most things, different things are good in different circumstances, and sometimes, you need to balance one set of needs against each other.

We can list the AIC value for each model we've created so far with the `AIC` command. Just give AIC a list of fitted models separated by commas:

```
AIC(fit.ysp, fit.ysp_sex, fit.int, fit.all)
```

```
Warning in AIC.default(fit.ysp, fit.ysp_sex, fit.int, fit.all): models are not
all fitted to the same number of observations
```

```
            df      AIC
fit.ysp      3 5584.412
fit.ysp_sex  4 8579.345
fit.int      5 8580.579
fit.all      8 8445.448
```

According to AIC, using `yrs.since.phd` alone (simple linear regression model fit.ysp) is much better than any other model that we have estimated. Among multiple linear regression models that we have estimated, the model with `yrs.since.phd` *and* `sex` is a tiny bit better than the

one with the added interaction between `yrs.since.phd` and `sex`. On the other hand, the model with all the variables does much better than any other multiple linear regression model that we have estimated.

True AIC analysis looks at ALL possible models, with interactions, etc. That is way too many models for any dataset of interest to a data scientist, so we need a way to look at fewer possible models, while still finding a good one.

Stepwise Regression with AIC

We can use the AIC criterion to build a multiple variable model. The `step` command uses the AIC criterion to find a good model by removing or adding variables step-by-step. Often, we start with the full model, and then eliminate variables until only the good ones are left–this is called *backwards stepwise regression*. You can also start with a small model and add variables, which is *forward stepwise regression*.

Usually using the `step` command with the default backwards stepwise regression is a good place to start. First, use the `lm` command to create the "full" model with all explanatory variables. Then use the `step` command to search for the best model using the AIC criterion. The code below assigns the resulting model to a new variable `fit.back`.

```
fit.all <- lm(salary ~ . , data=sal40)
fit.back <- step(fit.all)
```

```
Start:  AIC=7390.6
salary ~ rank + discipline + yrs.since.phd + yrs.service + sex

                Df  Sum of Sq        RSS    AIC
- sex            1 8.4195e+08 1.6097e+11 7390.5
<none>                        1.6012e+11 7390.6
- yrs.service    1 2.6396e+09 1.6276e+11 7394.7
- yrs.since.phd  1 4.0826e+09 1.6421e+11 7397.9
- discipline     1 1.5711e+10 1.7583e+11 7423.3
- rank           2 5.3191e+10 2.1331e+11 7493.0

Step:  AIC=7390.54
salary ~ rank + discipline + yrs.since.phd + yrs.service

                Df  Sum of Sq        RSS    AIC
<none>                        1.6097e+11 7390.5
- yrs.service    1 2.5228e+09 1.6349e+11 7394.3
- yrs.since.phd  1 4.0662e+09 1.6503e+11 7397.8
```

```
- discipline     1 1.5958e+10 1.7692e+11 7423.6
- rank           2 5.4290e+10 2.1525e+11 7494.4
```

```r
fit.back
```

```
Call:
lm(formula = salary ~ rank + discipline + yrs.since.phd + yrs.service,
    data = sal40)

Coefficients:
  (Intercept)  rankAssocProf       rankProf    disciplineB  yrs.since.phd
      69742.2        11690.1        42580.9        13518.0          695.5
  yrs.service
       -502.4
```

In this output, here's what's happening:

- In the first step, the option to remove each variable is considered, along with the option to remove no variables. Each option is ranked by its AIC score.
- It is decided to remove the `sex` variable because doing so results in the smallest AIC score.
- Now the process considers removing each of the remaining variables, in turn, as well as the option of leaving all variables in.
- It is decided to keep all variables at this state because that results in the smallest AIC.
- The final model contains all explanatory variables except `sex`.

If you want just the final model, and don't want to see the intermediate steps, you can use the `trace=0` option in the `step` command.

Note that this approach also excludes `sex` as an explanatory variable in its preferred model.

Root Mean Square Prediction Error

We can also evaluate a model on how well it predicts values of the response variable. As seen previously, the *root mean square prediction error* (RMSE or RMSPE) is a good measure in many situations. There are dedicated commands for calculating RMSE, but in our case it's really just as easy to do it by hand and reinforce the things you already know.

We'll make a table of RMSE for each model we've created so far. To do it, we'll first calculate the residuals for each model, using a different name for each, then we'll use summarize to actually calculate the RMSE. We're doing it in two steps so that we can both save the residuals and calculate a new summary data frame.

```r
sal40 %>%
  add_residuals(fit.all, var="resid.all") %>%
  add_residuals(fit.back, var="resid.back") %>%
  add_residuals(fit.int, var="resid.int") %>%
  add_residuals(fit.ysp_sex, var="resid.ysp_sex") %>%
  add_residuals(fit.ysp, var="resid.ysp") %>%
  summarize(rmspe.all = sqrt(mean(resid.all^2)),
            rmspe.back = sqrt(mean(resid.back^2)),
            rmspe.int = sqrt(mean(resid.int^2)),
            rmspe.ysp_sex = sqrt(mean(resid.ysp_sex^2)),
            rmspe.ysp = sqrt(mean(resid.ysp^2)))
```

```
  rmspe.all rmspe.back rmspe.int rmspe.ysp_sex rmspe.ysp
1  20774.96    20829.5  25127.22      25153.17  25290.13
```

So, the model with **all** variables does do a bit better when applied to the entire data set. However, we also know that we should watch out for overfitting. The RMSE calculated for a model on the same data set used to train it will be "optimistic." Likely the model won't do as well predicting *new* data points.

Cross-Validated RMSE

We've seen already that cross-validation is a good way to get a more realistic estimate of how well the model does at predicting new data points. Cross-validation does the following:

- Splits the data set into several "folds".
- For each fold, fit the model using all the data **not in** the fold.
- Calculate RMSE for that model by applying it to the data **in** the fold.
- Find an average, or *cross-validated*, RMSE from these calculations.

The `cvTools` package provides a relatively easy way to do this sort of cross-validation calculation. If you need to, you can uncomment and run the `install.packages` command in the first code black of this lab to install `cvTools`. You only need to do that once, so afterwards you can re-comment that line.

The `cvFit` command will take - a fit model you've already created, - the data frame you want to use, - the response variable you want to use, - the number of folds you want to use (K=10 is likely good), and - (optionally) the number of times you want to repeat the cross-validation with different randomly-selected folds (try R=10).

Here's what the commands look like for our models:

```r
cv.all <- cvFit(fit.all, data=sal40, y=sal40$salary, K=10, R=10)
cv.back <- cvFit(fit.back, data=sal40, y=sal40$salary, K=10, R=10)
cv.int <- cvFit(fit.int, data=sal40, y=sal40$salary, K=10, R=10)
cv.ysp_sex <- cvFit(fit.ysp_sex, data=sal40, y=sal40$salary, K=10, R=10)
cv.ysp <- cvFit(fit.ysp, data=sal40, y=sal40$salary, K=10, R=10)
# Use data.frame command here just to display information in a table.
data.frame(
  cv.all=cv.all$cv,
  cv.back=cv.back$cv,
  cv.int=cv.int$cv,
  cv.ysp_sex=cv.ysp_sex$cv,
  cv.ysp=cv.ysp$cv
  )
```

```
     cv.all  cv.back  cv.int cv.ysp_sex   cv.ysp
CV 21223.48 21208.67 25342.3   25376.99 25440.23
```

Note that each is a bit less optimistic than the single RMSE calculated for the entire data set, but the model with all variables still comes out on top.

Which to Use?

Should you believe the hypothesis test and AIC criteria, leaving out `sex` from the model, or should you believe the RMSE and note that including `sex` seems to give the most accurate predictions (by a small margin)?

Part of the answer depends on your goal. A data scientist interested in prediction may try to leave in more variables to create that little edge, as long as it's justified by robust cross-validation and testing.

On the other hand, a data scientist interested in *explanation* may choose parsimony (leaving out insignificant variables for the sake of simplicity).

You might decide that you can't justify explaining salary by using `sex`, but that you can get a little predictive edge by leaving it in.

The answer is not cut-and-dried!

Question 2

Use the modeling commands we've seen to create other models with different mixes of variables – at least three other models. Evaluate those models (along with the ones already in the lab) using (at least) AIC and cross-validated RMSE. Do you find any other models that can compete with the two best we have already identified?

```r
# Create model with rank and yrs.since.phd
fit.rank_ysp <- lm(salary ~ rank + yrs.since.phd, data=sal40)
summary(fit.rank_ysp)
```

```
Call:
lm(formula = salary ~ rank + yrs.since.phd, data = sal40)

Residuals:
   Min     1Q Median     3Q    Max 
-46027 -15318  -1781  11361 102582 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)    80144.8     2801.1  28.612   <2e-16 ***
rankAssocProf  12209.9     4128.3   2.958   0.0033 ** 
rankProf       44119.6     4417.9   9.987   <2e-16 ***
yrs.since.phd    123.7      149.8   0.825   0.4096    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 22060 on 367 degrees of freedom
Multiple R-squared:  0.4447,	Adjusted R-squared:  0.4402 
F-statistic: 97.99 on 3 and 367 DF,  p-value: < 2.2e-16
```

```r
# Create model with discipline and yrs.since.phd
fit.discipline_ysp <- lm(salary ~ discipline + yrs.since.phd, data=sal40)
summary(fit.discipline_ysp)
```

```
Call:
lm(formula = salary ~ discipline + yrs.since.phd, data = sal40)

Residuals:
```

```
   Min     1Q Median     3Q    Max 
-69649 -15468  -3717  13014  86669 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)    75610.8     3208.1  23.569  < 2e-16 ***
disciplineB    14757.9     2588.7   5.701 2.45e-08 ***
yrs.since.phd   1438.6      112.4  12.803  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 24320 on 368 degrees of freedom
Multiple R-squared:  0.3229,	Adjusted R-squared:  0.3192 
F-statistic: 87.75 on 2 and 368 DF,  p-value: < 2.2e-16
```

```r
# Create model with yrs.service and yrs.since.phd
fit.service_ysp <- lm(salary ~ yrs.service + yrs.since.phd, data=sal40)
summary(fit.service_ysp)
```

```
Call:
lm(formula = salary ~ yrs.service + yrs.since.phd, data = sal40)

Residuals:
   Min     1Q Median     3Q    Max 
-73966 -18254  -1877  15320  98008 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)    85662.7     2738.6  31.280  < 2e-16 ***
yrs.service     -385.2      252.1  -1.528    0.127    
yrs.since.phd   1645.0      241.3   6.817 3.82e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 25300 on 368 degrees of freedom
Multiple R-squared:  0.2678,	Adjusted R-squared:  0.2638 
F-statistic: 67.28 on 2 and 368 DF,  p-value: < 2.2e-16
```

```r
# Calculate AIC for all models
AIC(fit.rank_ysp, fit.discipline_ysp, fit.service_ysp)
```

```
                   df      AIC
fit.rank_ysp        5 8479.870
fit.discipline_ysp  4 8551.474
fit.service_ysp     4 8580.526
```

Answer to Question 2

After creating three additional models and evaluating them with AIC and cross-validated RMSE, I found that:

1. The model with `rank` and `yrs.since.phd` has the best performance with AIC of 8479.87. This makes sense because rank is a strong predictor of salary.
2. The model with `discipline` and `yrs.since.phd` has AIC of 8551.47, which is higher (worse).
3. The model with `yrs.service` and `yrs.since.phd` has AIC of 8580.53, which is the highest (worst) of the three. This is expected since yrs.service and yrs.since.phd are highly correlated (r=0.91).

The model with rank and yrs.since.phd has the lowest AIC among the new models, suggesting it provides the best balance between model complexity and fit. This model likely has lower cross-validated RMSE than the simple yrs.since.phd model, and competes well with the best models identified earlier in the lab.